

leaklens: A Unified Toolkit for Benchmark Contamination Detection

Shreyas K Chandrabhas

Abstract

Benchmark contamination detection methods are individually well studied – n-gram overlap against pretraining corpora, Min-K% Prob membership inference [Shi et al., 2023], the zlib-entropy perplexity baseline [Carlini et al., 2021], guided completion, and order-sensitivity probes in the tradition of the GPT-3 contamination appendix [Brown et al., 2020] – but no maintained, standard implementation packages them behind one interface, so every paper that wants to check for contamination either reimplements a detector or waves hands. We present **leaklens**, a plugin-based toolkit implementing six such detectors: n-gram overlap against the public infini-gram index [Liu et al., 2024], guided completion, order canary, Min-K% Prob, a perplexity-vs-compressibility gap, and paraphrase gap. Detectors are explicit about their own applicability: logprob-based methods (Min-K% Prob, perplexity gap, paraphrase gap) require a local model and report `applicable=False` rather than silently failing against API-only judges, and every report card states which detectors ran, which were skipped, and why. We validate the toolkit with a real-hardware calibration pilot: two LoRA adapters fine-tuned on disjoint 15-item subsets of GSM8K (one “contaminated”, one “clean”) using a local Qwen2.5-0.5B model, then scored with every logprob- and completion-based detector on the contaminated subset. Guided completion, Min-K% Prob, and the perplexity-compressibility gap all separate the fine-tuned-on model from the not-fine-tuned-on model cleanly (AUC 0.92, 1.00, and 1.00 respectively, on this small $n = 15$ pilot); a fourth detector, order-sensitivity, scored at chance, and we trace this to a genuine methodological artifact in the calibration’s own training-data design (documented in Section 3) rather than to a flaw in the detection method itself. All six of the toolkit’s dataset benchmark adapters (MMLU, GSM8K, HumanEval, ARC, HellaSwag, TruthfulQA) are verified against the live HuggingFace `datasets-server` API rather than assumed from memory. **leaklens** is available at <https://github.com/Shreyaskc/leaklens>.

1 Introduction

Benchmark contamination – a model having seen a benchmark’s items, or close paraphrases of them, during training – is one of the central validity threats in LLM evaluation. A growing body of work has produced individual detection methods: substring/n-gram overlap against known pretraining corpora, membership-inference statistics computed from a model’s own token log-probabilities [Shi et al., 2023], extraction and compressibility-based probes [Carlini et al., 2021], and behavioral probes that ask whether a model can complete a benchmark item verbatim or recover its canonical ordering [Brown et al., 2020]. Each method has a paper; none has a maintained, general-purpose implementation that a researcher auditing a new benchmark–model pair can simply install and run. The result is that contamination checks, when they appear in papers at all, are usually a single ad hoc n-gram search rather than the fuller battery the literature already supports.

leaklens packages six such methods behind one API, `leaklens.audit(model, benchmark)`, which returns a report card listing, per detector, whether it ran, its aggregate score, and (for detectors that were not applicable to the given model) why. The library is deliberately an aggregator, not a competitor to the underlying methods: every detector’s docstring and every report card cites the

paper the method comes from, so leaklens is cited alongside the primitives it wraps, not instead of them.

2 The leaklens Toolkit

2.1 Plugin interface

Every detector implements a common `Detector` interface with a `requires_logprobs` flag and a `run(model, benchmark)` method. A `ModelInterface` abstraction exposes `generate()` (supported by any model, local or API) and `token_logprobs()` (supported only by local models with weight access). Before running a detector, `leaklens.audit()` checks `detector.applicable(model)`; if a logprob-only detector is asked to run against an API-only model, the report card records `applicable=False` with an explicit reason rather than crashing or silently omitting the row. This matters for reproducibility: a report card should show what was *not* checked, not just what was.

2.2 Detectors

N-gram overlap. Queries the public infini-gram API [Liu et al., 2024] for exact-match counts of a benchmark item’s text in a named pretraining-corpus index (Dolma, the Pile, C4, or RedPajama). This detector is deliberately model-independent – it answers whether an item’s text is present in a *public corpus*, not whether the specific model under audit was trained on that exact corpus, and its report-card entry states this scope explicitly. The endpoint and response schema were verified against the live API during development rather than assumed.

Guided completion. Prompts the model with a prefix of a benchmark item (a configurable fraction of its length) and scores the completion’s similarity to the true continuation via a normalized sequence-matching ratio. High-fidelity verbatim reproduction of a continuation the model was never shown at inference time is a memorization signal in the tradition of training-data-extraction attacks [Carlini et al., 2021].

Order canary. For each adjacent pair of items in a benchmark’s canonical published order, prompts the model with the tail of item i and scores whether the completion matches the head of item $i+1$ (the real next item) versus a randomly reassigned “fake next” item (a shuffled control). A model that predicts the real next item reliably better than the shuffled control has memorized the benchmark’s exact sequence, independent of item content – a content-independent contamination signal in the tradition of GPT-3’s own contamination appendix [Brown et al., 2020].

Min-K% Prob. Following Shi et al. [2023], computes the mean log-probability of the $k\%$ least-likely tokens in an item’s text. Text a model has memorized tends to have a less-negative worst-case token probability than genuinely unseen text, since exact repetition raises even the model’s least confident predictions.

Perplexity-compressibility gap. Following the zlib-entropy baseline from Carlini et al. [2021], divides a model’s cross-entropy on an item by the item’s zlib-compressed byte length. Raw perplexity alone confounds memorization with an item simply being low-entropy; dividing by a model-independent compressibility measure controls for that confound.

Paraphrase gap. Compares a model’s cross-entropy on an item’s original text versus a semantically-equivalent paraphrase. A model that finds the exact original phrasing much easier than an equivalent paraphrase is showing surface-form memorization rather than task competence. This detector’s logic is implemented and tested, but leaklens does not yet ship pre-generated, human-checked paraphrases for its six built-in benchmarks – a documented data-construction gap, not a silent omission; callers may supply their own paraphrases via a field on each benchmark item.

Detector	AUC
Guided completion	0.92
Min-K% Prob	1.00
Perplexity-compressibility gap	1.00
Order canary	0.50

Table 1: Calibration pilot AUC-ROC, $n = 15$ items, one base model, one fine-tune pass per condition. Ngram overlap and paraphrase gap are not included – see Section 3.1.

2.3 Benchmark adapters

Built-in adapters wrap six widely used benchmarks – MMLU, GSM8K, HumanEval, ARC, HellaSwag, and TruthfulQA – as sources of items with a fixed, canonical ordering (required for the order-canary detector to be meaningful). Every adapter’s HuggingFace dataset repository ID, config name, split name, and column names were checked directly against the live `datasets-server` API during development, not assumed from prior familiarity with these datasets; the checks are re-run as part of the test suite so upstream schema drift is caught rather than silently producing empty or malformed items.

3 Calibration Pilot

The scientific claim a contamination detector makes – “this score indicates the model saw this item during training” – is only as good as its calibration against known ground truth. We ran a small, fast calibration pilot entirely on local hardware (an Apple M4 Pro, no rented or cloud GPU): two LoRA adapters were fine-tuned from the same base model, `Qwen2.5-0.5B-Instruct` (4-bit), on two disjoint 15-item subsets of GSM8K test questions – one subset (“contaminated”) the model was repeatedly fine-tuned on, one (“clean”) it never saw. Each of the four model-dependent detectors was then run against the contaminated subset’s 15 items using both fine-tuned models, and we computed the AUC-ROC of each detector’s score at distinguishing the contaminated (member) model from the clean (non-member) model on the same items.

Table 1 reports the results. Three detectors – guided completion, Min-K% Prob, and the perplexity-compressibility gap – cleanly separate the two models on this pilot. The fourth, order canary, scored at chance ($\text{AUC} \approx 0.50$), with both models’ completions at the relevant prompt boundary being an empty string. We traced this to a real artifact of this calibration’s own training-data construction, not a flaw in the underlying method: each individual GSM8K item was included in the fine-tuning data as its own training example (implicitly teaching the model “predict end-of-sequence after this exact tail”), while adjacent-item pairs were *also* included as continuous training examples (teaching “continue into the next item”) at the exact same text boundary. These two training signals conflict at inference time with no way for the model to disambiguate which one applies, and it appears to default to stopping. An isolated calibration design – training only on the continuous concatenated sequence, without the individual-item examples – would be needed to calibrate order canary cleanly, and is left for a follow-up run. We report this null result and its diagnosis rather than omit the detector or present only the three detectors that happened to work.

3.1 Scope of the calibration

Two of the six detectors are not covered by this calibration, for principled reasons rather than oversight. N-gram overlap queries a fixed, public pretraining-corpus index and is architecturally independent of any model under audit (see Section 2.2); a local LoRA fine-tune on GSM8K items has no bearing on whether those items independently appear in a large public web corpus, so this pilot’s fine-tuning design cannot calibrate it. Paraphrase gap requires paraphrase data this release does not yet ship (Section 2.2).

We also note plainly that this is a single small pilot – one base model, $n = 15$ items, one fine-tuning run per condition – reported as an indicative first pass, not the larger multi-model calibration suite (with true/false positive rates across many models and content types) that a mature calibration study would require.

4 Validation

The package includes 74 unit and integration tests (99% statement coverage), including tests that exercise the real, live infini-gram API response schema (not a hand-written mock of an assumed schema), the real MLX-based logprob computation validated against expected model behavior (a repeated sentence scores a substantially higher log-probability on its second occurrence than its first; character-level gibberish scores reliably lower than natural language), and the calibration pipeline described in Section 3, which was itself run twice: a first run produced NaN training loss partway through fine-tuning, traced to a single training example roughly an order of magnitude longer than the rest of its batch destabilizing gradients; the fix (splitting that one long example into shorter adjacent-pair examples) is reflected in the released calibration script.

5 Limitations

The calibration pilot in Section 3 is small-scale by design (one base model, $n = 15$ items) and should be read as a sanity check that leaklens’s detectors behave in the expected direction, not as a definitive true/false-positive-rate characterization suitable for setting a universal decision threshold. The boilerplate-phrase and artifact-signature detection this toolkit’s broader design contemplates is not yet part of the released detector set. Paraphrase gap’s data gap (Section 2.2) means it is currently usable only by callers who supply their own paraphrases. Order canary’s null result in this pilot (Section 3) means we do not yet have a clean calibration data point for that detector; the diagnosis suggests a fix, but the fix itself has not yet been run.

6 Ethics Statement

This work is a measurement toolkit; it does not collect, redistribute, or generate new human-subject data. The infini-gram queries send benchmark item text (already public benchmark content) to a third-party public API; no private or user data is involved. The calibration pilot fine-tunes a small open-weight model on public GSM8K questions already released under an open license, entirely on local hardware.

7 Availability

leaklens is available at <https://github.com/Shreyaskc/leaklens> under the MIT license.

8 Conclusion

We presented `leaklens`, a plugin-based contamination-detection toolkit unifying six methods from the literature behind one interface, with explicit per-detector applicability reporting and six verified benchmark adapters. A local-hardware calibration pilot confirms that three of four model-dependent detectors cleanly separate a fine-tuned-on item set from a held-out one on a small sample, and traces the fourth detector’s null result to a specific, fixable artifact of the calibration’s own design rather than papering over it.

References

- Tom B Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33:1877–1901, 2020.
- Nicholas Carlini, Florian Tramèr, Eric Wallace, Matthew Jagielski, Ariel Herbert-Voss, Katherine Lee, Adam Roberts, Tom B Brown, Dawn Song, Úlfar Erlingsson, Alina Oprea, and Colin Raffel. Extracting training data from large language models. In *30th USENIX Security Symposium (USENIX Security 21)*, pages 2633–2650, 2021.
- Jiacheng Liu, Sewon Min, Luke Zettlemoyer, Yejin Choi, and Hannaneh Hajishirzi. Infini-gram: Scaling unbounded n-gram language models to a trillion tokens. *arXiv preprint arXiv:2401.17377*, 2024.
- Weijia Shi, Anirudh Ajith, Mengzhou Xia, Yangsibo Huang, Daogao Liu, Terra Blevins, Danqi Chen, and Luke Zettlemoyer. Detecting pretraining data from large language models. *arXiv preprint arXiv:2310.16789*, 2023.